

Lectura 4

Errores numéricos

Dr. Luciano Ponzellini Marinelli
lucianoponzellini@uca.edu.ar

Facultad de Química e Ingeniería
Rosario, Argentina

Notas de clase
1 abril 2025



INTRODUCCIÓN

ERROR DE TRUNCAMIENTO

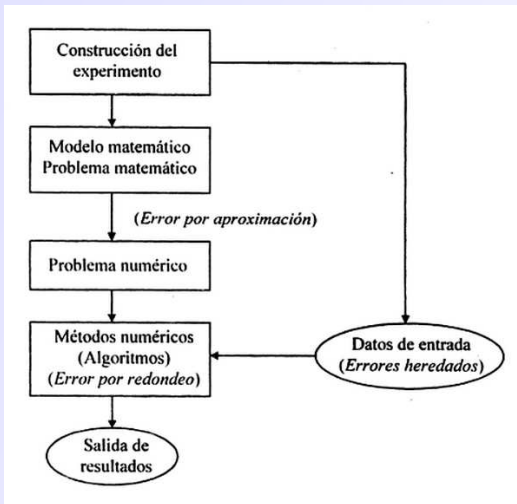
ERROR DE REDONDEO

ERROR TOTAL

PROPAGACIÓN DE ERRORES

INTRODUCCIÓN

Errores en aplicaciones



Errores en aplicaciones

- **Errores en datos de entrada (data uncertainty):** cometidos en los valores numéricos de datos de entrada. Son '*sistemáticos*' (imprecisión de los aparatos de medición) o '*experimentales*' (apreciación del observador y otras causas)
- **Simplificaciones en la construcción del modelo (model errors):** simplificaciones que hacemos al fenómeno físico para representarlo como un modelo matemático y poder resolverlo
- **Errores causados por la computadora o el hombre (blunders):** debido al mal funcionamiento de una computadora o por imperfecciones humanas
- **Errores numéricos (numerical errors):** los introducimos al usar un algoritmo en computadora, y están asociados al '*algoritmo*' (tiempo finito) y al '*hardware*' (espacio finito)

Conceptos básicos en el análisis del error

- **Exactitud (accuracy):** refiere a la proximidad de un valor calculado respecto al valor verdadero
- **Precisión (precision):** refiere a la proximidad de un valor particular calculado respecto a otros calculados
- **Cifras significativas:** dígitos distintos de cero de un número leyendo el número de izquierda a derecha
- **Overflow:** fenómeno numérico cuando en un espacio de memoria no puede almacenarse un número positivo (negativo) por ser mayor (menor) que el mayor (menor) número representable en esa memoria
Ejemplo: en MATLAB/Octave: *-realmax* y *realmax*
- **Underflow:** fenómeno numérico cuando dentro de la memoria no se puede almacenar un número positivo muy chico por ser menor al menor número representable en memoria.
Ejemplo: en MATLAB/Octave: *-realmin* y *realmin*

Exactitud versus precisión

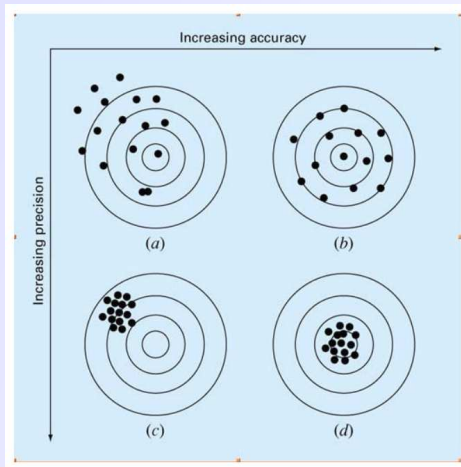


Figura: Representación esquemática de exactitud y precisión crecientes

¿Cómo medimos la exactitud?

Sea x_e el valor exacto de un número real y x_a su aproximación:

Error absoluto. Diferencia entre el valor exacto y su aproximado:

$$\Delta x = x_e - x_a.$$

Error relativo. Cociente entre el error absoluto y el valor exacto:

$$E_x = \frac{\Delta x}{x_e} = \frac{x_e - x_a}{x_e}$$

siempre que $x_e \neq 0$.

Error relativo porcentual. Error relativo en porcentaje:

$$E_x^{\%} = E_x * 100.$$

¿Cómo medimos la precisión en los errores?

Precisión. Diremos que x_a es una aproximación a x_e con d cifras decimales significativas, si d es el mayor número natural tal que:

$$\frac{|x_e - x_a|}{|x_e|} < \frac{1}{2} \times 10^{-d}.$$

EJEMPLOS:

- $x_e = 3,141592$ y $x_a = 3,14$:

$$|\Delta_x| = |x_e - x_a| = 0,001592$$

$$|E_x| = \frac{|x_e - x_a|}{|x_e|} = 0,000507 \mapsto E_x\% = 0,0507\%$$

$$|E_x| < 0,005 = \frac{1}{2} \times 10^{-2} \mapsto d = 2$$

Ejemplos de cálculo de precisión

- $y_e = 1000000$ y $y_a = 999997$:

$$|\Delta_y| = |y_e - y_a| = 3$$

$$|E_y| = \frac{|y_e - y_a|}{|y_e|} \approx 0,000003 \mapsto E_y\% \approx 0,0003\%$$

$$|E_y| < 0,000005 = \frac{1}{2} \times 10^{-5} \mapsto d = 5$$

- $z_e = 0,000012$ y $z_a = 0,000009$:

$$|\Delta_z| = |z_e - z_a| = 0,000003$$

$$|E_z| = \frac{|z_e - z_a|}{|z_e|} = 0,25 \mapsto E_z\% = 25\%$$

$$|E_z| < 0,5 = \frac{1}{2} \times 10^0 \mapsto d = 0$$

Ejemplos de cálculo en MATLAB/Octave/Scilab

```
>> xe=3.141592  
>> xa=3.14  
>> ex=abs(xe-xa)/abs(xe)  
ex= 5.067494e-04 d = 2 cifras significativas
```

```
>> ye=1000000  
>> ya=999997  
>> ey=abs(ye-ya)/abs(ye)  
ey= 3.0e-06 d = 5 cifras significativas
```

```
>> ze=1.2e-05  
>> za=9.0e-06  
>> ez=abs(ze-za)/abs(ze)  
ez=0.25 d = 0 cifras significativas
```

Notación científica normalizada (decimal)

Definición. Si $x \in \mathbb{R}$ podemos escribirlo como:

$$x_{10} = 0.d_1 d_2 d_3 \dots d_k \dots \times 10^n$$

$$d_1 = \{1, 2, \dots, 8, 9\}$$

$$d_j = \{0, 1, \dots, 8, 9\}$$

$$j \geq 2$$

- $d_1 d_2 d_3 \dots d_k$ se conocen como **mantisa**
- n como **exponente**

EJEMPLO:

$$9700000000 = 0.97 \times 10^{10}$$

$$732,5051 = 0.7325051 \times 10^3$$

$$-0,005612 = -0.5612 \times 10^{-2}$$

ERROR DE TRUNCAMIENTO

Tipos de errores numéricos: truncamiento

Error de truncamiento. Son errores debidos a la interrupción de un proceso matemático antes de su terminación (tiempo finito de corrida de un algoritmo)

EJEMPLOS:

① $f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0+h) - f(x_0)}{h}$ (cociente incremental)

$f'(x_0) \approx \frac{f(x_0+h) - f(x_0)}{h}$ para cierto h 'chico'

② $\{x_n\}_{n=0}^{\infty}$ tal que $x_n \xrightarrow{n \rightarrow \infty} r$ (proceso iterativo)
 $\{x_0, x_1, \dots, x_n\}$ tal que $x_n \approx r$ para cierto n 'grande'

③ $f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!} (x-a)^k$ (serie infinita)

$f(x) \approx f(a) + \frac{f'(a)}{1!} (x-a) + \frac{f''(a)}{2!} (x-a)^2 + \dots + \frac{f^{(n)}(a)}{n!} (x-a)^n$
 para cierto n 'grande'

Fórmula del error de Lagrange de la Serie de Taylor

- El desarrollo de Taylor de $f(x) \in C_{[b,c]}^{n+1}$ en a puede expresarse:

$$f(x) = \sum_{k=0}^n \frac{f^{(k)}(a)}{k!} (x-a)^k + \frac{f^{(n+1)}(\xi)}{(n+1)!} (x-a)^{(n+1)}$$

donde ξ está entre 0 y x en el intervalo (b, c)

- El primer término se conoce como *polinomio de Taylor* : $T_n(x, a)$
- El segundo es el *error de Taylor (forma de Lagrange)* : $E_n(x, a)$
- Este error de truncamiento puede ser controlado acotando:

$$|E_n(x, a)| = |f(x) - T_n(x, a)| = \left| \frac{f^{(n+1)}(\xi)}{(n+1)!} (x-a)^{n+1} \right|$$

Error de truncamiento: serie de Taylor

- El desarrollo de Taylor de $f(x) = e^x$ en $a = 0$ es:

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} = \sum_{k=0}^n \frac{x^k}{k!} + \frac{e^{\xi} x^{n+1}}{(n+1)!}$$

donde ξ está entre 0 y x .

Si estamos suficientemente cerca, la suma parcial 'converge'

- El error de truncamiento puede ser controlado acotando:

$$|E_n(x, a)| = \left| e^x - \sum_{k=0}^n \frac{x^k}{k!} \right| = \left| \frac{e^{\xi} x^{n+1}}{(n+1)!} \right|$$

Obtenemos una estimación con un número finito de términos

Error de truncamiento: TaylorExp.m

```
% Script: TaylorExp
% Plotea, como funcion de n, el error relativo en la aproximacion de
Taylor de la funcion exponencial
%  $1 + x + x^2/2! + \dots + x^n/n!$  a  $\exp(x)$ .
clear, clc, close all;
nTerms = 50;
xx = [1 5 10 -1 -5 -10];
for i=1:length(xx)
    x = xx(i);
    f = exp(x)*ones(nTerms,1);
    s = 1;
    term = 1;
    ...
```

Error de truncamiento: TaylorExp.m

```

...
for k=1:nTerms
    term = x.*term/k;
    s = s+ term;
    err(k) = abs(f(k) - s);
end
relerr = err'/exp(x)';
subplot(2,3,i), semilogy(1:nTerms,relerr), grid on, hold on
ylabel('Error Relativo Suma Parcial.')
xlabel('Orden Suma Parcial')
title(sprintf('x = %5.2f',x))
end

```

Error de truncamiento: TaylorExp.m

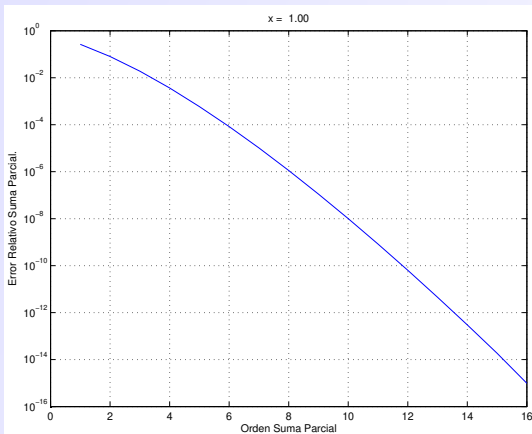


Figura: error relativo del Taylor de e^x en $x = 1$

¿Qué pasa en estos casos?

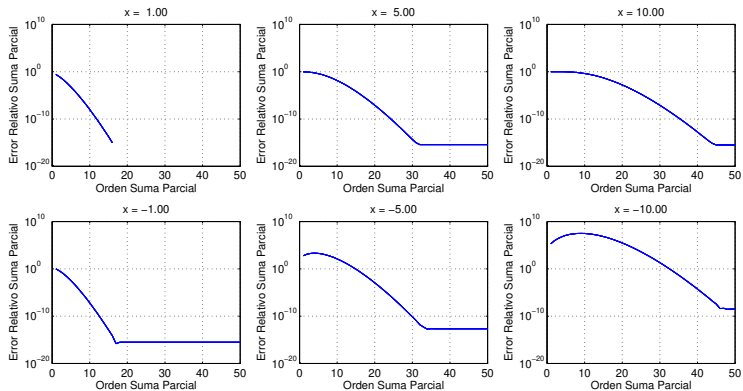


Figura: subplots de error relativo del Taylor de e^x para distintos x y variando n

Ejemplo error de truncamiento: TaylorExp.m

CONCLUSIONES:

- El ploteo muestra que la “*convergencia matemática*” no siempre es válida en la máquina
- Los errores no convergen a cero a medida que la serie aumenta la cantidad de términos
- El error parece acostarse en un valor pequeño.
Incorporando más términos, no hacemos diferencia
- El piso del error relativo depende de x :

$$Err_{x=-10} > 1 \times 10^{-10}$$

$$Err_{x=10} < 1 \times 10^{-15}$$

Hay otro error...

ERROR DE REDONDEO

23 / 54

¿Cuándo aparece el error de redondeo?

El **error de redondeo (roundoff error)** surge en:

- Almacenamiento de números: en máquina, sólo podemos representar una cantidad finita de cantidades finitas
- Grandes cálculos: si un proceso realiza una gran cantidad de cálculos, los errores de redondeo pueden ser significativos
- Cuando sumamos un número muy grande y un número muy chico: hay un corrimiento de la mantisa para poder sumar los números
- Cuando restamos números muy parecidos entre sí hay pérdidas de cifras significativas

Error de redondeo

EJEMPLO:

- Si representamos números en base 10 a binario (base 2):

$$11_{10} = 1011_2$$

$$0,1875_{10} = 0,0011_2$$

- Pero, ¿cómo representamos el decimal 0.1 en sistema binario?

$$\frac{1}{10} = 0,1_{10} = 0,00011001100110011..._2 = 0,\widehat{00011}_2$$

- Para almacenarlo tenemos que truncarlo. Hay dos formas:

- Redondeo Truncado
- Redondeo Simétrico

Redondeo Truncado

Redondeo truncado. Consiste en truncar el resultado de una operación al número de dígitos significativos que estemos utilizando.

- Si $x \in \mathbb{R}$ (decimal) normalizado con k cifras significativas:

$$x = \pm 0.d_1 d_2 \dots d_k d_{k+1} \dots \times 10^n$$

donde

$$d_1 = \{1, 2, \dots, 8, 9\}$$

$$d_j = \{0, 1, \dots, 8, 9\} \quad j > 1$$

- Entonces

$$fl_{trunc}(x) = \pm 0.d_1 d_2 \dots d_k \times 10^n$$

es su **representación de coma flotante por truncamiento**

Redondeo Simétrico

Redondeo simétrico. Consiste en aumentar en uno el último dígito si el primer dígito descartado está en $\{5, \dots, 9\}$ o dejarlo igual si el primer dígito descartado está en $\{0, \dots, 4\}$.

- En este caso

$$fl_{sim}(x) = \pm 0.d_1 d_2 \dots d_{k-1} d_k^* \times 10^n$$

es la **representación de coma flotante por redondeo de x** donde

$$d_1 = \{1, 2, \dots, 8, 9\}$$

$$d_j = \{0, 1, \dots, 8, 9\} \quad 1 < j \leq k$$

d_k^* se calcula redondeando el número $d_1 d_2 \dots d_k . d_{k+1} d_{k+2} \dots$ al número entero más próximo

Ejemplos de redondeos truncado y simétrico

- $x = 7/9 = 0,77777 \dots$ con 4 dígitos significativos:

$$fl_{trunc}(x) = 0.7777 \times 10^0$$

$$fl_{red}(x) = 0.7778 \times 10^0$$

- $y = 22/7 = 3,14285714285714285$ con 6 dígitos significativos:

$$fl_{trunc}(y) = 0.314285 \times 10^1$$

$$fl_{red}(y) = 0.314286 \times 10^1$$

- $z = 0,231599999$ con 8 dígitos significativos:

$$fl_{trunc}(z) = 0.23159999 \times 10^0$$

$$fl_{red}(z) = 0.23160000 \times 10^0$$

Ejemplo de redondeo: ZoomPoli.m

- Queremos plotear el polinomio:

$$p(x) = (x - 1)^6$$

en el intervalo $[0.9, 1.1]$

- Trabajando algebraicamente, equivale a:

$$p(x) = x^6 - 6x^5 + 15x^4 - 20x^3 + 15x^2 - 6x + 1$$

- El script ZoomPoli.m grafica este polinomio para entornos pequeños

$$[1 - \delta, 1 + \delta]$$

para $\delta = 0.1, 0.01, 0.008, 0.007, 0.005, 0.003$ cercanos a $x = 1$
usando la segunda expresión por extensión del polinomio:

$$[0.9, 1.1]$$

$$[0.99, 1.01]$$

$$[0.992, 1.008]$$

$$[0.993, 1.007]$$

$$[0.995, 1.005]$$

$$[0.997, 1.003]$$

Error de redondeo: ZoomPoli.m

```
% Script: ZoomPoli
% Plotea (x-1)^6 alrededor de x=1 con escala creciente
% pero evaluado via x^6 - 6x^5 + 15x^4 - 20x^3 + 15x^2 - 6x + 1
clear, clc close all;
k=0;
n=100;
for delta = [.1 .01 .008 .007 .005 .003 ]
    x = linspace(1-delta,1+delta,n)';
    y = x.^6 - 6*x.^5 + 15*x.^4 - 20*x.^3 + 15*x.^2 - 6*x + 1;
    k = k+1;
    subplot(2,3,k), plot(x,y,x,zeros(1,n)), grid on
    axis([1-delta 1+delta -max(abs(y)) max(abs(y))])
end
```

Error de redondeo: ZoomPoli.m

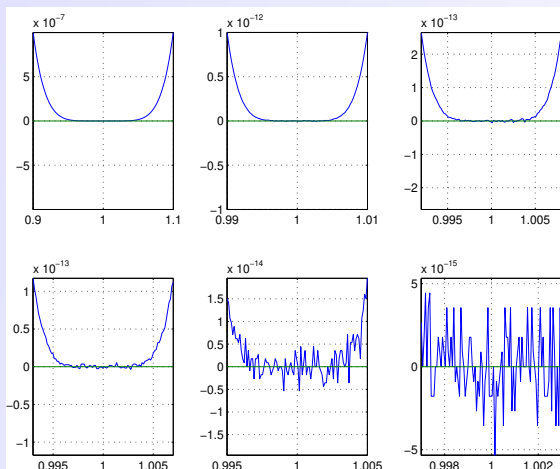


Figura: Error al plotear $(x-1)^6 = x^6 - 6x^5 + 15x^4 - 20x^3 + 15x^2 - 6x + 1$

Error de redondeo: ZoomPoli.m

CONCLUSIONES:

- El ploteo no parece un polinomio (ni siquiera es derivable)
- Acá no hay error de truncamiento (no truncamos un método ni tampoco iteramos)
- En las cercanías de $x = 1$, los valores de $p(x)$ son calculados restando números muy parecidos (potencias de x^6 y x^5)
- Estamos ante la acción de otro error numérico conocido como **pérdida de cifras significativas**

Veamos en detalle cómo funciona este error...

Pérdida de cifras significativas

- Sean las funciones

$$f(x) = x \left(\sqrt{x+1} - \sqrt{x} \right), \quad x \geq 0$$

y

$$g(x) = \frac{x}{\sqrt{x+1} + \sqrt{x}}, \quad x \geq 0$$

algebraicamente equivalentes (ejercicio)

- Vamos a comparar los resultados de calcular $f(500)$ y $g(500)$ usando 4 *cifras significativas con redondeo simétrico*

¿Debería obtenerse el mismo resultado en máquina?

Pérdida de cifras significativas

- Normalizando antes de cada operación, obtenemos:

$$\begin{aligned}
 f(500) &= 0,5000 \times 10^3 \left(\sqrt{0,5000 \times 10^3 + 0,1000 \times 10^1} - \sqrt{0,5000 \times 10^3} \right) \\
 &= 0,5000 \times 10^3 \left(\sqrt{0,5000 \times 10^3 + 0,0010 \times 10^3} - \sqrt{0,5000 \times 10^3} \right) \\
 &= 0,5000 \times 10^3 \left(\sqrt{0,5010 \times 10^3} - \sqrt{0,5000 \times 10^3} \right) \\
 &= 0,5000 \times 10^3 (22,38\textcolor{red}{30292855} \dots - 22,36\textcolor{red}{06797749} \dots) \\
 &= 0,5000 \times 10^3 (0,2238 \times 10^2 - 0,2236 \times 10^2) \\
 &= 0,5000 \times 10^3 (0,0002 \times 10^2) \\
 &= 0,5000 \times 10^3 (0,2000 \times 10^{-1}) \\
 &= 0,1000 \times 10^2 = 0,1000 \times 10^2 = 10
 \end{aligned}$$

Pérdida de cifras significativas

$$\begin{aligned}
 g(500) &= \frac{0,5000 \times 10^3}{\sqrt{0,5000 \times 10^3 + 0,1000 \times 10^1} + \sqrt{0,5000 \times 10^3}} \\
 &= \frac{0,5000 \times 10^3}{\sqrt{0,5000 \times 10^3 + 0,0010 \times 10^3} + \sqrt{0,5000 \times 10^3}} \\
 &= \frac{0,5000 \times 10^3}{\sqrt{0,5010 \times 10^3} + \sqrt{0,5000 \times 10^3}} \\
 &= \frac{0,5000 \times 10^3}{22,3830292855 \dots + 22,3606797749 \dots} \\
 &= \frac{0,5000 \times 10^3}{0,2238 \times 10^2 + 0,2236 \times 10^2} = \frac{0,5000 \times 10^3}{0,4474 \times 10^2} \\
 &= 1,11747575636 \dots \times 10^1 \\
 &= 0,1117 \times 10^2 = 11,17
 \end{aligned}$$

Pérdida de cifras significativas

- El valor 'exacto' es 11.174755300747198..., sin embargo:

$$f(500) = 10$$

$$g(500) = 11,17$$

- La respuesta de $g(500)$ tiene menor error absoluto que $f(500)$.

$$\Delta_{f(500)} \simeq 1,17475530075$$

$$\Delta_{g(500)} \simeq 4,75530074720 \times 10^{-3}$$

$$E_{f(500)} \simeq 1,05125818788 \times 10^{-1}$$

$$E_{g(500)} \simeq 4,25539586256 \times 10^{-4}$$

$$E_{f(500)}^{\%} \simeq 10,51 \%$$

$$E_{g(500)}^{\%} \simeq 0,04 \%$$

Pérdida de cifras significativas

- Definimos las funciones f y g en línea en MATLAB/Octave:

```
>> f=@(x) x*(sqrt(x+1)-sqrt(x))  
f = @(x) x*(sqrt(x+1)-sqrt(x))  
  
>> f(500)  
ans = 11.174755300746853  
  
>> g=@(x) x/(sqrt(x+1)+sqrt(x))  
g = @(x) x/(sqrt(x+1)+sqrt(x))  
  
>> g(500)  
ans = 11.174755300747199
```

ERROR TOTAL

Error numérico total

El **Error numérico total** es:

Error de truncamiento + Error de redondeo

- Si **disminuyo el Error de truncamiento** ↘
entonces el **Error de redondeo aumenta** ↗
- Si **aumento el Error de truncamiento** ↗
entonces el **Error de redondeo disminuye** ↘

*Disminuir una componente de error
conduce a un incremento en la otra componente
Por tanto hay que buscar una **solución de compromiso***

Error total en un método numérico

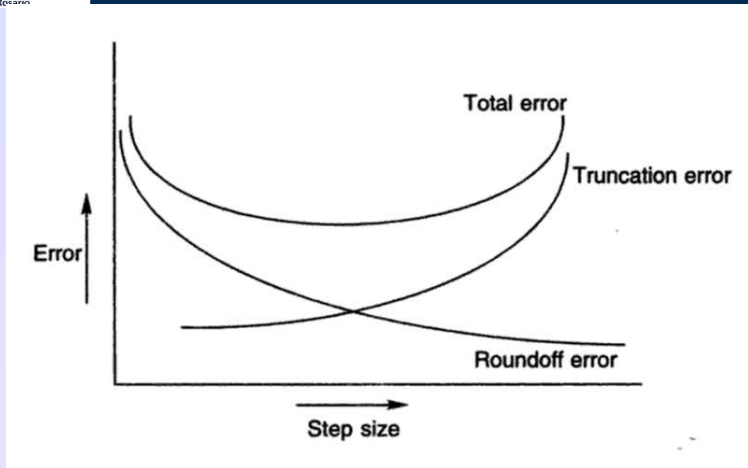


Figura: dependencia del Error total del tamaño del paso $h = 1/n$ (step size)

Pérdida de cifras significativas: SerieSeno.m

- Consideremos la serie de Taylor de $f(x) = \text{sen}(x)$ en $a = 0$:

$$\begin{aligned} \text{sen}(x) &= \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{(2k+1)!} \\ &= \sum_{k=0}^n \frac{(-1)^k x^{2k+1}}{(2k+1)!} + \frac{f^{(2n+3)}(\xi) x^{2n+3}}{(2n+3)!} \end{aligned}$$

donde ξ está entre 0 y x

- El error (de Lagrange) puede acotarse de la forma:

$$|E_{2n+3}(x, a)| = \left| \frac{f^{(2n+3)}(\xi) x^{2n+3}}{(2n+3)!} \right| \leq \frac{|x|^{2n+3}}{(2n+3)!}$$

El valor del Error de truncamiento depende de x y del número de términos de la sumatoria.

Pérdida de cifras significativas: SerieSeno.m

```
function ssum = SerieSeno(x,tol,n)
% SerieSeno.m: Evalúa la representación en serie de la función SENO
% ssum = SerieSeno(x)
% ssum = SerieSeno(x,tol)
% ssum = SerieSeno(x,tol,n)
%
% ENTRADA:
% x: argumento de la función SENO
% tol: (optional) tolerancia para sumatoria (por defecto: tol = 5e-9)
% La serie termina cuando  $\text{abs}(T\_k/S\_k) < \text{tol}$ , donde:
% T_k: k-ésimo término y S_k suma incluyendo el k-ésimo término
% n: (optional) máximo número de terminos (por defecto: n = 15)
%
% RESULTADO:
% ssum: valor de la serie al cabo de n terminos o al satisfacer la tolerancia
...
```

Pérdida de cifras significativas: SerieSeno.m

```

if nargin < 2,    tol = 5e-9;
end
if nargin < 3,    n = 15;
end
term = x; % Inicializa la serie
ssum = term;
fprintf('Valor aproximado de la función seno (%f)\n\n n término serie\ n',x)
fprintf(' %3d %11.3e %12.8f \ n',1,term,ssum)
for k=3:2:(2*n-1)
    term = -term .* x.*x./(k.*(k-1)); % Próximo término en la serie
    ssum = ssum + term;
    fprintf(' %3d %11.3e %12.8f\n',k,term,ssum)
    if abs(term./ssum) < tol % Verifica convergencia
        break
    end
end
fprintf('\n Error Total con %d términos es %g \ n\n',(k+1)/2,abs(ssum-sin(x)))
end

```

Pérdida de cifras significativas: SerieSeno.m

```
>> sum = SerieSeno(pi/4,5e-9,100)
```

Valor aproximado de la función seno (0.785398)

n	termino	serie
1	7.854e-01	0.78539816
3	-8.075e-02	0.70465265
5	2.490e-03	0.70714305
7	-3.658e-05	0.70710647
9	3.134e-07	0.70710678
11	-1.757e-09	0.70710678

Error Total incluyendo 6 términos es 6.92801e-12

```
sum = 0.707106781179619
```

Pérdida de cifras significativas: SerieSeno.m

```
>> sum = SerieSeno(pi/2,0,5)
```

Valor aproximado de la función seno (1.570796)

n	término	serie
1	1.571e+00	1.57079633
3	-6.460e-01	0.92483223
5	7.969e-02	1.00452486
7	-4.682e-03	0.99984310
9	1.604e-04	1.00000354

Error Total con 5 términos es 3.54258e-06

```
sum = 1.000003542584286
```

Pérdida de cifras significativas: SerieSeno.m

```
>> >> sum = SerieSeno(pi/2,0,15)
Valor aproximado de la función seno (1.570796)
```

n	término	serie
1	1.571e+00	1.57079633
3	-6.460e-01	0.92483223
5	7.969e-02	1.00452486
7	-4.682e-03	0.99984310
9	1.604e-04	1.00000354
11	-3.599e-06	0.99999994
13	5.692e-08	1.00000000
15	-6.688e-10	1.00000000
17	6.067e-12	1.00000000
19	-4.377e-14	1.00000000
21	2.571e-16	1.00000000
23	-1.254e-18	1.00000000
25	5.156e-21	1.00000000
27	-1.812e-23	1.00000000
29	5.507e-26	1.00000000

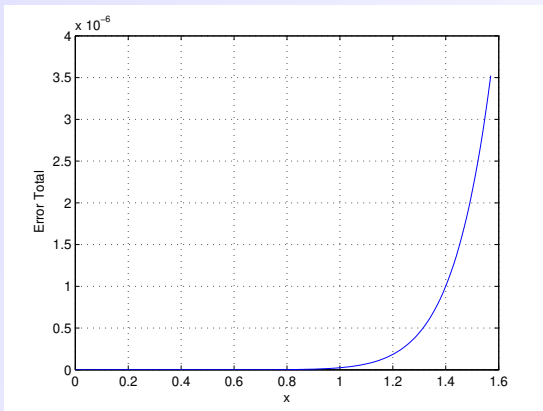
Error Total con 15 términos es 2.22045e-16

sum = 1.0000000000000000

Pérdida de cifras significativas: SerieSeno.m

En caso de utilizar 5 términos no nulos en la sumatoria:

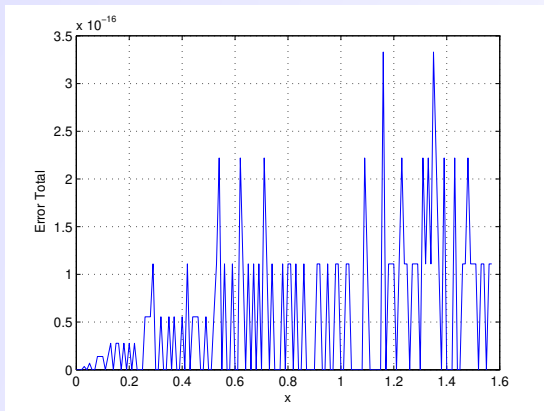
$$|E_{11}(x)| \leq \frac{|x|^{11}}{11!} \leq \frac{|\pi/2|^{11}}{11!} \approx 3,5988 \times 10^{-6}$$



Pérdida de cifras significativas: SerieSeno.m

En caso de utilizar 15 términos no nulos en la sumatoria:

$$|E_{31}(x)| \leq \frac{|x|^{31}}{31!} \leq \frac{|\pi/2|^{31}}{31!} \approx 1,4611 \times 10^{-28}$$



Pérdida de cifras significativas: sinserie.m

CONCLUSIONES:

- Para valores chicos de x necesitamos pocos términos para una “buena” aproximación
- Con 5 términos no nulos y valores de x mayores a 1 (aprox.), el error numérico total (truncamiento y redondeo) aumenta rápidamente cuando $x \rightarrow \pi/2$. El error máximo es $3.6e-06$ (aprox.) en consonancia con la cota del error teórico de truncamiento de Taylor expresado anteriormente
- Con 15 términos observamos un fenómeno numérico de oscilación que aumenta hasta un valor máximo de $3.3e-16$ (aprox.) mientras que la cota del error teórico de truncamiento de Taylor expresado antes es del orden de 10^{-28}
- Conclusión: el error de redondeo controla el comportamiento

PROPAGACIÓN DE ERRORES

Propagación del error en la suma

Veamos como se propaga el error de representación numérica en una cadena de operaciones sucesivas (pueden ser muchas).

- Sean los números exactos x e y con valores aproximados $\tilde{x}$, $\tilde{y}$, y los errores δ_x , δ_y respectivamente. Esto es:

$$x = \tilde{x} + \delta_x, \quad y = \tilde{y} + \delta_y.$$

- Queremos calcular el error δ_{x+y} :

$$x + y = (\tilde{x} + \delta_x) + (\tilde{y} + \delta_y) = (\tilde{x} + \tilde{y}) + \underbrace{(\delta_x + \delta_y)}_{\delta_{x+y}}.$$

- Por lo tanto:

El error absoluto de la suma de dos (o más magnitudes) es la suma de los errores absolutos de dichas magnitudes.

Propagación del error en el producto

- Sean x e y reales no nulos:

$$x = \tilde{x} + \delta_x = \tilde{x} \left(1 + \frac{\delta_x}{\tilde{x}} \right), \quad y = \tilde{y} + \delta_y = \tilde{y} \left(1 + \frac{\delta_y}{\tilde{y}} \right),$$

suponiendo $\tilde{x} \neq 0$ y $\tilde{y} \neq 0$.

- Queremos calcular el error $\delta_{x \cdot y}$ al aproximar el producto:

$$x \cdot y = \tilde{x} \tilde{y} \left(1 + \frac{\delta_x}{\tilde{x}} \right) \left(1 + \frac{\delta_y}{\tilde{y}} \right) = \tilde{x} \tilde{y} \left(1 + \frac{\delta_x}{\tilde{x}} + \frac{\delta_y}{\tilde{y}} + \underbrace{\frac{\delta_x \delta_y}{\tilde{x} \tilde{y}}}_{\approx 0} \right).$$

- Entonces

$$x \cdot y \approx \tilde{x} \tilde{y} \left(1 + \frac{\delta_x}{\tilde{x}} + \frac{\delta_y}{\tilde{y}} \right).$$

El error relativo del producto de dos (o más magnitudes) es aproximadamente igual a la suma de los errores relativos.

Bibliografía

- **[Apos1984]** T.M. Apostol, *"Calculus"*, Vol. 1, Ed. Reverté, Buenos Aires, 1984.
- **[KiCh1994]** D. Kincaid - W. Cheney, *"Análisis Numérico. Las matemáticas del cálculo científico"*, Addison-Wesley Iberoamericana, Delaware, 1994.
- **[MaFi2000]** J.M. Mathews - K.D. Fink, *"Métodos Numéricos con MATLAB"*, 3era Edición, Pearson Prentice Hall, Madrid, 2000.
- **[Mole2015]** C.B. Moler, *"Numerical computing with MATLAB"*, SIAM, Philadelphia, 2015.
- **[SPMK2023]** J.A. Semitiel, L. Ponzellini Marinelli & A.I. Kurdobrin, *Notas de Clases de Matemática Aplicada de Ingeniería Eléctrica e Ingeniería Electrónica*, FCEIA, UNR, 2023.
- **[SiSo2010]** J.W. Signorelli, J. Sorribas, *"Notas de Clases de Informática Aplicada y Métodos Computacionales"*, FCEIA, UNR, 2010.